ML4T Study Guide

Tailored for Traders & Software Engineers — Intuition over Proofs

> **Exam Flags** are marked with 🎯 throughout. These map directly to question types seen on practice exams.

Table of Contents

1. Machine Learning Foundations (Probabilistic ML Ch. 1)
2. AI & Big Data in Investments — Overview (Handbook Ch. 1)
3. Deep Learning Basics — Probability Foundations (DL Foundations Ch. 2.1–2.2)
4. Alternative Data & AI in Investment Research (Handbook Ch. 2)
5. Statistical Learning — Intro & Regression (ISL Ch. 1, 2.2, 3.1–3.3)
6. Tree Methods & Ensemble Learners (ISL Ch. 8.1–8.2)
7. Deep Learning — Why It Works (DL Foundations Ch. 1)
8. Regularization & Model Selection (ISL Ch. 3.5 + Ch. 6)
9. Hedge Funds — Structure & Trading (What Hedge Funds Really Do Ch. 2 & 4)
10. AI in Trade Execution & Factor Investing (Handbook Ch. 7)
11. Hedge Fund Strategies — Valuation & CAPM (What Hedge Funds Really Do Ch. 5 & 7)
12. Portfolio Management, EMH & Data Quirks (What Hedge Funds Really Do Ch. 8, 9, 10 & 12)

> ⚠️ **Topics from lectures (not in these readings — supplement from course notes):**
> - **Reinforcement Learning details**: Markov Decision Processes (MDP), Q-learning algorithm, Dyna/Dyna-Q (model-based RL), R-max
> - **Technical indicators**: Bollinger Bands, momentum indicators, moving averages
> - **Derivatives**: Options (calls, puts, straddle, butterfly spreads), leverage
> - **The Big Short**: CDS, CDO, synthetic CDO, mortgage-backed securities (MBS)

---

1. Machine Learning Foundations

Source: Probabilistic Machine Learning — Book 1, Chapter 1

What is Machine Learning?

Tom Mitchell's classic definition: "*A computer program learns from experience E with respect to task T and performance measure P, if performance at T improves with E.*"

Trader/SE translation: ML is function approximation under uncertainty. You have historical data (E), you want to predict prices or signals (T), and you measure how good your predictions are (P). The "probabilistic" approach means outputs are distributions — not single guesses — which is critical when sizing positions or managing risk.

Three Types of ML

🎯 **Exam Flag: Know the definitions and goals of all three.**

Type	What You Have	What You Learn	Trading Example
Supervised	Labeled (X, y) pairs	f(X) → y	Predict next-day returns from features
Unsupervised	Only X, no labels	Structure/patterns in p(X)	Cluster stocks by behavior; factor discovery
Reinforcement	Reward signals	Policy π: state → action	Q-learning to decide Buy/Hold/Sell

🎯 **Exam Flag: RL is NOT given the correct answer — it learns from rewards only.**
- Think of it as "learning with a critic (thumbs up/down)" vs supervised learning = "learning with a teacher (correct answer each step)"
- The RL cycle: **Sense** → **Think** → **Act** (state observation → policy evaluation → action)

Supervised Learning Deep Dive

Classification vs Regression

- **Classification**: output y is a discrete label (e.g., Buy/Sell/Hold)
- **Regression**: output y is a continuous value (e.g., predicted return = +2.3%)

Key vocabulary:
- **Features / covariates / predictors** = inputs X (e.g., P/E ratio, momentum, volume)
- **Labels / targets / responses** = outputs y
- **Design matrix**: N×D matrix where rows = examples, columns = features (N = samples, D = features)

Empirical Risk Minimization (ERM)

The goal: find model parameters θ that minimize average loss on training data:

```
θ̂ = argmin_θ (1/N) Σ loss(y_n, f(x_n; θ))
```

Common loss functions:
- **Classification**: 0-1 loss (misclassification rate), cross-entropy
- **Regression**: MSE = mean squared error = $(1/N) \Sigma (y_n - \hat{y}_n)^2$

Trader intuition: ERM is just "fit the model to your historical P&L." The danger is overfitting to noise in the data.

Maximum Likelihood Estimation (MLE)

When fitting probabilistic models, minimize **negative log-likelihood (NLL)**:

```
NLL(θ) = -(1/N) Σ log p(y_n | f(x_n; θ))
```

For Gaussian output distributions, minimizing NLL = minimizing MSE.

Overfitting vs Underfitting

🎯 **Exam Flag: The U-shaped test error curve — know which direction means overfitting.**

- **Underfitting** (too simple): high error on BOTH train and test
- **Overfitting** (too complex): low train error, high test error → large **generalization gap**
- **Just right**: minimize test error

```
Generalization Gap = Test Error - Train Error
```

In practice:
- Split data: **training set** (fit model) → **validation set** (tune hyperparameters) → **test set** (final evaluation only)
- Never use test set during model selection — it becomes biased

🎯 **Exam Flag: kNN overfits as k decreases (k=1 = memorize training data). DT overfits as leaf_size decreases.**

No Free Lunch Theorem

No single model is best for all problems. Model choice depends on **inductive bias** — assumptions baked into the model architecture. Always trial-and-error with cross-validation.

SE implication: Don't cargo-cult architectures. A linear model may beat a neural net on small tabular financial data.

Two Types of Uncertainty

🎯 **Exam Flag: Know the difference.**

Type	Also Called	Cause	Can We Reduce It?
Epistemic	Model uncertainty	Not enough data / wrong model	Yes — more data, better model
Aleatoric	Data uncertainty	Irreducible noise in the world	No

Trading context: Aleatoric uncertainty = market microstructure noise. Epistemic uncertainty = regime changes not in your training data.

Unsupervised Learning Key Concepts

- **Clustering** (e.g., K-means): group stocks by similar behavior patterns
- **Dimensionality reduction** (e.g., PCA): extract latent factors from high-dim data (think: Fama-French factors from hundreds of stock returns)
- **Self-supervised learning**: create proxy tasks from unlabeled data (e.g., predict masked features)
- **Evaluating unsupervised models is hard** — no ground truth. Common approach: use learned features as inputs to a downstream supervised task

---

Reinforcement Learning Overview

🎯 **Exam Flag: All of these RL components appear on exams.**

- **Agent**: the decision-maker (your trading algorithm)
- **Environment**: the market
- **State (s)**: current situation (e.g., Bollinger band value, current position, momentum)
- **Action (a)**: what to do (Buy / Sell / Hold)
- **Reward (r)**: feedback signal (e.g., daily P&L)
- **Policy π(s)**: maps state → action (or probability distribution over actions)
- **Goal**: find π that maximizes cumulative discounted reward

Key challenge: **credit assignment problem** — which past action caused this reward? (like chess: you only find out you won/lost at the end)

Data Handling

🎯 **Exam Flag: Missing data fill order.**

Missing data recommended approach: fill forward, then fill back (forward fill propagates last known price; backfill handles gaps at the start)

One-hot encoding: convert categorical variables (e.g., sector = "Tech") into binary vectors. Required for linear models.

Feature engineering: transform raw inputs into informative features — polynomial expansion, log-returns, moving averages, etc.

---

2. AI & Big Data in Investments — Overview

Source: Handbook of AI and Big Data Applications in Investments, Chapter 1 — "On Machine Learning Applications in Investments" (Robeco)

Why ML Beats Linear Models in Finance

The fundamental advantage: **ML can capture nonlinear and interaction effects** that traditional linear models (OLS, factor models) miss.

Interaction effect example: Earnings quality and reported earnings interact. If an accounting red flag is present, earnings don't matter — the stock drops anyway. A linear model treats earnings and red flags independently and misses this.

Nonlinear effect example: A company's credit default swap (CDS) spread has a nonlinear relationship with equity returns (equity = call option on firm assets). Linear models can't capture this curve.

Empirical evidence (Gu, Kelly, Xiu 2020):
- Neural network long-short decile strategy: Sharpe ratio 1.35 (value-weighted) vs. 0.61 for OLS
- Interaction effects are the **primary driver** of ML outperformance

---

Three Unique Challenges of ML in Finance

🎯 **Exam Flag: These challenges explain why ML is harder to apply in finance than in other domains.**

1. Low Signal-to-Noise Ratio

Financial returns compress many causal factors (macro, micro, behavioral, positioning) into a single +/- number. No single input is a reliable predictor. Compare: Netflix recommendation = high signal (genre preference); stock return prediction = low signal.

Implication: Simple models often beat complex ones when data is limited. More data matters more than better architecture.

2. Small Data Problem

- Even 100 years of US monthly data = ~1,200 observations per stock
- Compare to Netflix (billions of data points), image recognition (millions of labeled examples)
- Alternative data (social media, news) is more abundant but still limited
- **Consequence:** Harder to do proper cross-validation; more prone to overfitting

3. Non-Stationarity (Adaptive Markets)

Financial markets learn and adapt. When a successful strategy is discovered and published, it gets arbitraged away (McLean & Pontiff 2016). The rules governing returns change over time:
- Regime changes (policy, regulation, market participants)
- Unlike protein folding (static physics) — markets are man-made and evolve

Consequence: Training data from the past may not reflect current market dynamics. Models trained in stable periods may fail during volatility spikes.

Common Pitfalls and How to Avoid Them

🎯 **Exam Flag: Overfitting and data leakage are the most testable concepts here.**

Overfitting

- More severe in finance than other domains (small data + high model complexity)
- Standard defenses: cross-validation, regularization, early stopping, feature selection, ensembling, holdout data
- **Domain knowledge is your best defense**: use economic intuition to curate inputs. "CEO last-name first letter" is not a valid feature.
- Use **SHAP values** to verify feature relationships make economic sense (e.g., high leverage → higher crash probability)

Data Leakage

Two types:
1. **Leakage in data**: Test set information bleeds into training set (e.g., overlapping time windows). Fix: use strict chronological splits + a **gap period** between train/validation sets
2. **Leakage in features**: A feature is informative in backtesting but unavailable at production time (e.g., using an executive's undisclosed health status). Fix: ensure features are truly available point-in-time

🎯 **Exam Flag: Lookahead bias = using data before it was actually available. E.g., using Q3 earnings reported 40 days after quarter end AS IF available on the quarter-end date.**

Implementation Gap

- ML often finds alpha in **small/micro-cap stocks** — but these are illiquid and costly to trade at scale
- Shorting may be infeasible (no shares to borrow, high borrow costs)
- ML models tend to have **high turnover**, amplifying transaction costs
- Always verify backtested positions can realistically be traded

Explainability (the "Black Box" Problem)

- Asset owners demand attribution: *why* did the model make this trade?

- **SHAP (SHapley Additive exPlanations)**: measures marginal contribution of each feature to the prediction
- **Partial dependence response**: plot how model output changes as one feature varies; decompose into linear + nonlinear components
- This allows performance attribution: how much return came from linear factor exposure vs. interaction effects vs. nonlinear effects?

ML Applications in Finance (Practitioner Results)

Application	Setup	Key Result
Cross-sectional return prediction	Features: valuation, quality, momentum	ML significantly outperforms OLS; interaction effects explain most of the edge
Crash/distress prediction	Features: distance to default, volatility, beta	ML (logistic reg, RF, gradient boost) > traditional metrics (leverage, D2D)
Earnings forecast accuracy	Features: 56 financial statement variables	Nonlinear ML > linear ML > traditional models; **ensembling always helps**
NLP / Sentiment	Multi-language investment blogs	Word embeddings capture investment slang; useful for non-English markets

🎯 **Exam Flag:** "Ensembling is the closest thing to a free lunch in ML, much like diversification in investing." Ensembling almost always outperforms individual models.

Key Takeaways for Traders

1. ML works best when you have enough data and can validate truly out-of-sample
2. Interaction effects (how factors combine) matter more than nonlinearity alone
3. Feature engineering guided by economic intuition dramatically reduces overfitting
4. A well-explained 70% strategy often beats an unexplained 80% strategy (institutional credibility)
5. Markets are adaptive — strategies decay; plan for model refresh cycles

---

3. Deep Learning Basics — Probability Foundations

Source: Deep Learning: Foundations and Concepts, Chapter 2 (Sections 2.1–2.2)

This chapter lays the probabilistic foundation that underpins all ML models. Think of it as the math language that lets models express "I'm 70% sure this stock will rise."

Two Types of Uncertainty (Again — It's Important)

🎯 **Exam Flag: Epistemic vs. aleatoric uncertainty appears across multiple readings.**

Type	Cause	Can You Reduce It?	Trading Example
Epistemic (model uncertainty)	Limited data, wrong model	Yes — more data, better model	Model didn't see 2008-style crises
Aleatoric (data/noise uncertainty)	Irreducible stochasticity	No (or only with more data types)	True randomness in order flow

---

The Two Rules of Probability

These two rules + Bayes' theorem derive everything else in probability theory.

Sum Rule (marginalization):

```
p(X) = Σ_Y p(X, Y)    [discrete]
p(x) = ∫ p(x, y) dy   [continuous]
```

"*To get the probability of X alone, sum/integrate over all possible values of Y.*"

Product Rule:

```
p(X, Y) = p(Y|X) · p(X)
```

"*Joint probability = conditional × marginal.*"

Bayes' Theorem

Derived directly from the product rule. The most important formula in probabilistic ML:

```
p(Y|X) = p(X|Y) · p(Y) / p(X)
```

Components:
- **Prior** `p(Y)`: your belief BEFORE seeing data X
- **Likelihood** `p(X|Y)`: probability of seeing this data given Y is true
- **Evidence** `p(X)`: normalizing constant (ensures probabilities sum to 1)
- **Posterior** `p(Y|X)`: updated belief AFTER seeing data X

Trader intuition: Bayesian reasoning = updating your position conviction as new information arrives. Prior = your thesis before earnings; posterior = updated view after reading the call transcript.

Worked example (cancer screening = base rate problem):
- 1% of population has cancer (prior)
- Test is 90% accurate for cancer, 3% false positive rate
- If you test positive: only ~23% chance you actually have cancer
- **Key insight**: Low prior probability dominates. This is why rare event predictions (e.g., market crash) always have high false positive rates even with good models.

Probability Densities (Continuous Variables)

For continuous variables (like stock prices, returns), we use **probability density functions (PDFs)**:

```
P(a ≤ x ≤ b) = ∫_a^b p(x) dx
```

Properties:
- `p(x) ≥ 0` (non-negative)
- `∫ p(x) dx = 1` (integrates to 1)
- The probability of any exact single value = 0 (only intervals have non-zero probability)

Common distributions:
- **Uniform**: constant over interval — equal probability everywhere in range
- **Exponential**: `p(x|λ) = λ·exp(-λx)` — wait times, durations between events
- **Laplace**: like Gaussian but with heavier tails — good for modeling return distributions with fat tails
- **Gaussian (Normal)**: `N(x|μ, σ²)` — the workhorse of ML; arises from Central Limit Theorem

---

Expectations, Variance, and Covariance

These are the building blocks of risk measurement:

Quantity	Formula	What It Measures
Expectation E[f]	Σ p(x)f(x) or ∫ p(x)f(x)dx	Weighted average (mean return)
Variance var[x]	$E[x^2] - E[x]^2$	Spread around the mean (risk/volatility^2)
Covariance cov[x,y]	E[xy] - E[x]E[y]	How x and y move together

🎯 **Exam Flag: Independent variables have covariance = 0. But covariance = 0 does NOT always mean independent (unless Gaussian).**

Covariance in portfolio context:
- `cov[x,y] > 0`: stocks move together (correlated) → less diversification benefit

- `cov[x,y] < 0`: stocks move oppositely → natural hedge, reduces portfolio variance
- `cov[x,y] = 0`: uncorrelated → maximum diversification benefit for given expected return

The portfolio variance with two assets is:

```
var[portfolio] = w₁² var[x] + w₂² var[y] + 2·w₁·w₂·cov[x,y]
```

This is why **negative covariance** (anticorrelated assets) reduces portfolio risk.

---

4. Alternative Data & AI in Investment Research

Source: Handbook of AI and Big Data Applications in Investments, Chapter 2 — "Alternative Data and AI in Investment Research" (Goldman Sachs)

What Is Alternative Data?

Alternative data = any data in a nonstructured format (as opposed to traditional structured financial data like price, earnings, P/E ratios).

Examples of alternative data categories:
Category	Examples	Use Case
Geospatial	Satellite imagery, mobile foot traffic	Count cars in Walmart parking lots → predict sales
Sentiment / NLP	Social media, news, earnings call text	Gauge market sentiment before it moves prices
Web data	Search trends, web scraping, app reviews	Consumer demand signals, product sentiment
Transaction data	Credit card data, shipping records	Real-time consumer spending
Survey data	Purchasing manager surveys, consumer confidence	Forward-looking economic signals

---

Two Strategy Types and How They Use Alt Data Differently

🎯 **This connects to exam questions about IC, BR, and portfolio construction strategies.**

Strategy	Priority	Best Use of Alt Data
Systematic / Quantitative	Breadth (many positions)	High-frequency signals applied to thousands of stocks simultaneously (e.g., intraday news sentiment for HFT)
Fundamental / Active	Depth (fewer, higher-conviction positions)	Niche datasets for specific companies — builds a richer mosaic view (e.g., satellite images of a specific company's factories)

Investment horizon matters: Intraday news sentiment helps HFT but adds little value to a portfolio manager with a multi-month horizon.

The Most Underappreciated Part: Data Grunt Work

The most time-consuming and critical part of AI in investments is **not the model** — it's the **data preparation**:

1. **Data ingestion**: getting raw data into a usable format

2. **Data cleaning**: handling NaNs, outliers, formatting issues
3. **Data curation**: applying domain judgment — is this data source trustworthy? Are there survivorship biases? Is it point-in-time?
4. **Feature construction**: creating meaningful signals from raw data

> "Do not expect AI to provide insights if you do not understand the data you apply AI to."

Implication for traders/SEs: Building a robust data pipeline and understanding your data is more valuable than having the most sophisticated model.

The "Pull" Approach to AI in Research

Goldman Sachs's framework: **start from the investment question, not from the data.**

- **Wrong**: "We have satellite data. What investment question can we use this for?"
- **Right**: "We want to predict retail traffic. What data can help us answer that?"

This is called the "pull" approach: the investment thesis pulls in relevant data, rather than technology pushing data at analysts hoping it's useful.

Human + AI = Winning Combination

Neither data scientists nor investment analysts can succeed alone:
- **Data scientists** don't know which financial signals are economically meaningful or how to interpret them in context
- **Investment analysts** can't process hundreds of datasets or apply ML at scale
- **Collaboration**: analysts do "smell tests" on ML outputs; data scientists scale and automate analyst intuition

Bottom line: AI augments human judgment — it doesn't replace it. The "AI + HI (human intelligence)" model dominates.

Key Limitations in Finance (Again)

- Financial market dynamics are **inherently unstable and adaptive** (markets learn and react)
- Limited data means in-sample vs. out-of-sample splits are harder
- Some patterns are coincidental; domain knowledge distinguishes signal from noise
- AI return on investment (ROI) is hard to measure → this creates an unfair hurdle vs. traditional research

---

Practical Advice for Starting an AI Initiative

1. Hire 1-2 data scientists who are **genuinely interested in investment problems** (not just ML engineers)

2. Pair them with experienced investment analysts
3. Start with high-probability use cases (multi-team or repeated use cases)
4. Document failures, not just successes ("research graveyard" principle from Ch. 1)
5. Build infrastructure first: version control, data pipelines, reproducibility

---

5. Statistical Learning — Intro & Regression

Source: Introduction to Statistical Learning (ISL), Chapters 1, 2.2, 3.1–3.3

What is Statistical Learning?

Statistical learning = estimating an unknown function **f** such that **Y ≈ f(X) + ε**, where:
- **Y** = response/output (e.g., stock returns)
- **X** = predictors/features (e.g., volume, P/E ratio, sentiment score)
- **ε** = irreducible noise (always present; sets a floor on prediction error)

Two goals:
1. **Prediction** — get accurate ŷ; don't care about interpretability
2. **Inference** — understand *which* features matter and *how* (interpretability matters)

> Trader translation: Predicting next-day price move = prediction. Building a factor model to explain alpha = inference.

Regression vs. Classification

🎯 **Exam Flag: Know the difference.**

	Regression	Classification
Output	Continuous (price, return)	Categorical (buy/sell/hold, up/down)
Loss	MSE, RSS	Misclassification rate
Examples	Predict return magnitude	Predict direction

2.2 — Assessing Model Accuracy

Mean Squared Error (MSE)

Training MSE = average of (actual − predicted)^2 on training data.
Test MSE = average of (actual − predicted)^2 on held-out data.

> Key insight: Training MSE always decreases as model complexity increases. Test MSE forms a **U-shape** — the sweet spot is where test MSE is minimized.

🎯 **Exam Flag: Overfitting = model has low training MSE but high test MSE. The model memorized noise instead of signal.**

The Bias-Variance Tradeoff

🎯 **Exam Flag: This is one of the most tested concepts in ML.**

```
Expected Test MSE = Bias² + Variance + Irreducible Error
```

Term	What It Means	Trading Analogy
$Bias^2$	Error from wrong assumptions; how far off your model is on average	Using a linear model when relationship is curved
Variance	How much your model changes when you retrain on different data	Model that fits every tick of training data perfectly
Irreducible Error (Var(ε))	Noise that cannot be reduced no matter what	Random market noise; you cannot explain it

The tradeoff:
- **Flexible models** (low bias, high variance) — fit training data very well, but predictions vary wildly with new data → **overfit**
- **Simple/rigid models** (high bias, low variance) — consistent but systematically wrong → **underfit**
- **Optimal model** = balances both; minimizes total expected test MSE

> **kNN example:** Small K = very flexible (memorizes training data, high variance). Large K = smoother, more rigid (high bias). K=1 always gets training accuracy 100% but generalizes poorly.

🎯 **Exam Flag — kNN bias/variance direction:**
- Decrease K → more flexible → lower bias, higher variance → more overfitting
- Increase K → less flexible → higher bias, lower variance → more underfitting

---

3.1 — Simple Linear Regression

Model: $\mathbf{Y = \beta_0 + \beta_1 X + \varepsilon}$

- β_0 = intercept (predicted Y when X=0)
- β_1 = slope (change in Y per 1-unit change in X)

Fitted using **Ordinary Least Squares (OLS)** — minimizes **RSS (Residual Sum of Squares)**:

```
RSS = Σ(yᵢ - ŷᵢ)²
```

OLS coefficient estimates:

```
β̂₁ = Σ(xᵢ - x̄)(yᵢ - ȳ) / Σ(xᵢ - x̄)²
β̂₀ = ȳ - β̂₁ x̄
```

Measuring Fit Quality

RSE (Residual Standard Error) = average deviation of actual values from the regression line, in units of Y:

```
RSE = sqrt(RSS / (n-2))
```

R^2 (Coefficient of Determination) = proportion of variance in Y explained by X:

```
R² = (TSS - RSS) / TSS      where TSS = Σ(yᵢ - ȳ)²
```

- R^2 = 1: perfect fit; R^2 = 0: model explains nothing
- R^2 = 0.60 means 60% of variability in Y is explained by X

🎯 **Exam Flag: R^2 is between 0 and 1. High R^2 doesn't guarantee a good model (can still overfit).**

Statistical Significance

t-statistic = $\hat{\beta}_1$ / SE($\hat{\beta}_1$) — tests if β_1 is significantly different from 0.
p-value = probability of seeing this result by chance if $\beta_1 = 0$.
- p-value < 0.05 → statistically significant (feature matters)
- p-value > 0.05 → no evidence of relationship

95% Confidence Interval for β_1:

```
β̂₁ ± 2 × SE(β̂₁)
```

Interpretation: "We are 95% confident the true β_1 falls in this range."

3.2 — Multiple Linear Regression

Model: $\mathbf{Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon}$

Each $\hat{\beta}_i$ is interpreted as: *the average effect on Y of a one-unit increase in X_i, holding all other predictors fixed.*

F-statistic

🎯 **Exam Flag: The F-statistic tests whether ANY predictor is useful.**

```
F = [(TSS - RSS)/p] / [RSS/(n-p-1)]
```

- F >> 1 → at least one predictor is significant
- F ≈ 1 → no predictors explain Y above random chance
- Individual t-tests can be misleading with many predictors (false positives); F-test is the right overall test

Multicollinearity / Confounding

🎯 **Exam Flag: Classic ISL example — newspaper ads vs radio ads.**

In simple LR, newspaper advertising looks significant. But in multiple LR (adding radio), newspaper loses significance — because newspaper is *correlated with radio* (when radio spend is high, newspaper spend is also high, so newspaper was getting credit for radio's effect).

> **Trader lesson:** Correlated factors (e.g., momentum + short-term reversal) will steal significance from each other. Always check correlations before interpreting coefficients.

3.3 — Qualitative Predictors & Extensions

Dummy Variables (One-Hot Encoding)

For **binary qualitative predictor** (e.g., owns house: yes/no):

```
x = 1 if owns house, 0 otherwise
Y = β₀ + β₁x + ε
```

- β_0 = average Y for non-owners (baseline)
- $\beta_0 + \beta_1$ = average Y for owners
- β_1 = difference between groups

For **multi-level qualitative predictor** (e.g., region: East/South/West):
- Create **k-1 dummy variables** (one is the **baseline**)
- Always one fewer dummy than the number of levels
- Use F-test to test whether region matters at all

> Finance note: Sector, exchange, country, and rating category (investment-grade/HY) are common qualitative predictors encoded as dummies.

Interaction Terms (Synergy Effects)

Standard model assumes features act independently: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \varepsilon$

But what if the effect of X_1 depends on X_2? Add an **interaction term**:

```
Y = β₀ + β₁X₁ + β₂X₂ + β₃(X₁×X₂) + ε
```

Rewritten: $\tilde{\beta}_1 = \beta_1 + \beta_3 X_2$, so the slope on X_1 changes with X_2.

🎯 **Exam Flag: The hierarchical principle — if you include an interaction term $X_1 \times X_2$, you must also include the main effects X_1 and X_2, even if they are not individually significant.**

Qualitative × Quantitative interaction = different slopes for different groups:
- Without interaction: parallel regression lines (same slope, different intercepts)
- With interaction: different slopes — the groups respond differently to the predictor

> **Trading example:** The effect of earnings surprise on returns may be stronger for small-cap stocks than large-cap. That's an interaction between earnings surprise and market cap.

Polynomial / Non-linear Regression

When relationship is curved, add polynomial terms:

```
Y = β₀ + β₁X + β₂X² + ε  (still a linear model — linear in the coefficients!)
```

Diagnostic: If residual plot shows a U-shape or systematic pattern → add polynomial terms.

Key Formulas Summary

Formula	What it is
`Y = β₀ + β₁X + ε`	Simple linear regression
`RSS = Σ(yᵢ - ŷᵢ)²`	Residual sum of squares (minimize this for OLS)
`TSS = Σ(yᵢ - ȳ)²`	Total sum of squares (total variability in Y)
`R² = (TSS-RSS)/TSS`	Fraction of variance explained (0 to 1)
`RSE = sqrt(RSS/(n-2))`	Avg prediction error in Y units
`F = [(TSS-RSS)/p] / [RSS/(n-p-1)]`	Overall model significance
`MSE = Bias² + Variance + Irreducible`	Bias-variance decomposition

Exam Cheat Sheet — ISL Regression

- **Overfitting**: low training MSE, high test MSE — model has high variance
- **Underfitting**: both MSEs are high — model has high bias
- **R^2 near 1**: model explains most variance; R^2 near 0: model is useless
- **p-value < 0.05**: predictor is statistically significant
- **F close to 1**: no predictors matter; F >> 1: at least one does
- **Multicollinearity**: correlated predictors steal each other's significance
- **Dummy variables**: always k-1 dummies for k-level qualitative predictor
- **Interaction term**: slope of X_1 changes with X_2; always include main effects too
- **kNN**: decrease K → overfit (high variance); increase K → underfit (high bias)

---

6. Tree Methods & Ensemble Learners

Source: Introduction to Statistical Learning (ISL), Chapters 8.1–8.2

Decision Trees — The Intuition

A decision tree partitions the feature space into rectangular regions using a series of **if/else splits**, then predicts the mean (regression) or majority class (classification) in each region.

Example (Salary prediction):

```
Is Years < 4.5?
  → Yes: predict $165K (young players earn less)
  → No: Is Hits < 117.5?
        → Yes: predict $403K
        → No: predict $845K
```

Decision trees are like flowcharts — extremely interpretable, but can overfit easily.

8.1.1 — Regression Trees

Building process: Recursive Binary Splitting

At each step, choose the predictor X_j and cutpoint s that minimizes RSS:

```
Minimize: Σ(yᵢ - ŷ_R1)² + Σ(yᵢ - ŷ_R2)²
```

- **Greedy**: makes the locally best split at each step (not globally optimal)
- **Top-down**: starts with all data, progressively splits
- Predict: mean response of training data in each terminal node

Problem: A fully grown tree overfits — too many nodes, memorizes training noise.

Tree Pruning (Cost Complexity Pruning)

Solution: Grow a large tree, then prune it back. Use **cross-validation** to select the best subtree size.

Penalized criterion (like Lasso for trees):

```
Minimize: RSS + α × |T|
```

Where |T| = number of terminal nodes, α = complexity penalty.
- α = 0: keep full tree
- α increases: prune branches to reduce complexity
- Select α via K-fold cross-validation

🎯 **Exam Flag: Decreasing leaf_size (= more leaves) → more complex tree → more overfitting. This is the decision tree analog of decreasing K in kNN.**

8.1.2 — Classification Trees

For classification, can't minimize RSS — instead use **impurity measures**:

Measure	Formula	Interpretation
Gini Index	$G = \Sigma\, \hat{p}_{mk}(1 - \hat{p}_{mk})$	Small when nodes are pure (one class dominates)
Entropy	$D = -\Sigma\, \hat{p}_{mk} \log(\hat{p}_{mk})$	Also small when nodes are pure

Both Gini and entropy measure **node purity** — how "mixed" the classes are in a region.

> Trader translation: You want each leaf of the tree to clearly say "buy" or "sell", not a 50/50 mix.

When pruning (evaluating final tree quality), use classification error rate instead.

8.1.3-4 — Decision Trees: Pros & Cons

Pros	Cons
Very interpretable (like human decision-making)	High variance — small data changes = very different tree
Handles qualitative predictors natively (no dummy vars)	Generally lower accuracy than ensemble methods
Visual and intuitive	Greedy splits miss globally optimal solutions
Works with mixed variable types	

🎯 **Exam Flag: Single decision trees have HIGH variance — key motivation for ensemble methods.**

8.2 — Ensemble Methods

> **Core idea:** Combine many weak learners (mediocre individual trees) to produce one powerful model. All three main methods reduce variance by averaging many trees.

8.2.1 — Bagging (Bootstrap Aggregation)

Problem: A single tree has high variance.
Solution: Average many trees trained on different bootstrap samples.

```
Algorithm:
1. Take B bootstrap samples from training data (sample with replacement)
2. Fit a full (unpruned, deep) tree to each sample
3. Average predictions (regression) or majority vote (classification)
```

Why it works: Averaging B independent estimates with variance σ^2 gives variance σ^2/B — much lower than any single tree.

🎯 **Exam Flag: Bagging reduces VARIANCE. Each tree has high variance/low bias; averaging reduces variance without increasing bias.**

Out-of-Bag (OOB) Error

Each bootstrap sample uses ~2/3 of observations. The remaining ~1/3 (the **OOB observations**) are not used to fit that tree — they serve as a built-in validation set.

- Compute OOB prediction for each observation across trees where it was held out
- **OOB error ≈ leave-one-out cross-validation error** (computationally efficient)
- No need for a separate validation set

Variable Importance

Bagging loses the single tree's interpretability, but you can measure **variable importance**:
- For each predictor, compute total RSS decrease (regression) or Gini decrease (classification) across all B splits
- High decrease = important variable

8.2.2 — Random Forests

Problem with Bagging: If one predictor is very strong, all B trees will use it at the top split → trees are highly correlated → averaging correlated trees doesn't reduce variance much.

Solution: At each split, randomly sample **m predictors** (not all p) as split candidates.

```
Typical choice: m ≈ √p    (for p total predictors)
```

- Each split considers only m randomly chosen features
- Strong predictors get "shut out" on many splits → other predictors get a chance
- Trees become **decorrelated** → averaging them gives bigger variance reduction

🎯 **Exam Flag: Random Forest key parameter = m (# features considered at each split). m = p → equivalent to bagging. m = √p → standard random forest.**

Compared to bagging:
- Random forests generally achieve lower test error
- Especially valuable when many correlated predictors exist

8.2.3 — Boosting

Different philosophy: Don't train trees independently — train them **sequentially**, each one correcting the errors of the previous.

```
Algorithm (Gradient Boosting):
1. Start: ŷ = 0, residuals rᵢ = yᵢ for all i
2. Repeat B times:
   a. Fit a small tree fᵇ to the RESIDUALS (not original Y)
   b. Update: ŷ ← ŷ + λ × fᵇ  (small step in direction of improvement)
   c. Update residuals: rᵢ ← rᵢ - λ × fᵇ(xᵢ)
3. Final model = Σ λ × fᵇ(x)
```

Three tuning parameters:

Parameter	Role	Effect if too large
B (# trees)	Total iterations	Overfitting (boosting CAN overfit unlike bagging)
λ (shrinkage)	Learning rate (e.g. 0.01)	Very slow learning, needs large B
d (depth/# splits)	Tree complexity per step	Overfitting

🎯 **Exam Flag: Boosting learns SLOWLY — fits residuals of the current model. Unlike bagging, boosting CAN overfit if B is too large.**

Key difference from bagging:
- Bagging: parallel, independent trees; averaging reduces variance
- Boosting: sequential, dependent trees; slowly reduces BIAS by correcting residuals

---

Ensemble Methods Summary

🎯 **Exam Flag: Ensemble methods reduce overfitting (lower variance), but do NOT make training or prediction faster.**

Method	How Trees Built	What's Random	Reduces	Can Overfit?
Bagging	Parallel, bootstrap samples	Data (bootstrap)	Variance	No (with enough B)
Random Forest	Parallel, bootstrap samples	Data + m features at each split	Variance (more than bagging)	No
Boosting	Sequential, fit residuals	Nothing (deterministic)	Bias	Yes (if B too large)

ML Algorithm Comparison (exam-critical)

🎯 **Exam Flag: These comparison questions appear frequently.**

Algorithm	Training Time	Query Time	Overfits when...
kNN	None (lazy learner — stores training data)	Slow (searches all data)	K decreases (K=1 = 100% train accuracy)
Decision Tree	Slow (recursive splitting)	Fast (follow splits)	leaf_size decreases (more leaves)
Linear Regression	Moderate (matrix operations)	Very fast (dot product)	Adding too many features (p ≈ n)

🎯 **Ensemble learner facts for exam:**
- Ensemble learners have LESS overfitting than single learner
- Ensemble learners are NOT faster (they average many models)
- Ensemble learners are NOT more interpretable

Exam Cheat Sheet — Trees & Ensembles

- **Decision tree splitting**: minimize RSS (regression) or Gini/entropy (classification) at each split
- **Tree pruning**: grow large tree, then apply cost-complexity penalty + CV to find best subtree
- **Bagging**: B bootstrap trees, averaged → reduces variance, NOT bias
- **Random Forest**: bagging + random feature subset $m \approx \sqrt{p}$ → decorrelates trees → better than bagging
- **Boosting**: sequential trees fit to residuals, shrinkage λ → reduces bias slowly; CAN overfit
- **OOB error**: uses the ~1/3 of data not in bootstrap sample to estimate test error
- **Variable importance**: total RSS/Gini decrease across all splits for each predictor

---